Faculty of Technology and Engineering

Computer Science and Engineering

Practical

Academic Year	:	2025-26	Semester	:	6
Course code	:	CSE312	Course name	:	Design of language processing

Practical - 7

1. Objective:

Develop a program computes the First and Follow sets for all non-terminal symbols in for the below given grammar.

S → A B C | D
A → a | ε
B → b | ε
C → (S) | c
D → A C

2. Program Code:

```
#include <iostream>
#include <map>
#include <set>
#include <vector>
#include <string>
#include <algorithm>

using namespace std;

// Grammar:
// S -> A B C | D
// A -> a | epsilon
// B -> b | epsilon
// C -> ( S ) | c
// D -> A C

// Non-terminals: S, A, B, C, D
// Terminals: a, b, c, (, ), $
// epsilon represented as ""

map<string, vector<vector<string>>> grammar;
map<string, set<string>> firstSets;
map<string, set<string>> followSets;
set<string> nonTerminals;
```

```
set<string> terminals;

bool isNonTerminal(const string& sym) {
  return nonTerminals.count(sym) > 0;
}

// Compute FIRST of a sequence of symbols (alpha)
// Returns true if epsilon is in FIRST(alpha)
bool computeFirstOfSequence(const vector<string>& seq, set<string>& result) {
  if (seq.empty() || (seq.size() == 1 && seq[0] == "eps")) {
    result.insert("eps");
    return true;
  }

  bool allHaveEpsilon = true;
  for (const string& sym : seq) {
    if (sym == "eps") {
      // epsilon symbol, continue
      continue;
    }
    if (!isNonTerminal(sym)) {
      // terminal
      result.insert(sym);
      allHaveEpsilon = false;
      break;
    } else {
      // non-terminal: add FIRST(sym) - {eps}
      for (const string& s : firstSets[sym]) {
        if (s != "eps") result.insert(s);
      }
      if (firstSets[sym].count("eps") == 0) {
        allHaveEpsilon = false;
        break;
      }
      // eps in FIRST(sym), continue to next symbol
    }
  }

  if (allHaveEpsilon) {
    result.insert("eps");
    return true;
  }
  return false;
}

void computeFirst() {
  // Initialize
  for (const string& nt : nonTerminals) {
    firstSets[nt] = {};
  }
```

```
  bool changed = true;
  while (changed) {
    changed = false;
    for (const string& nt : nonTerminals) {
      for (const vector<string>& prod : grammar[nt]) {
        set<string> tmp;
        computeFirstOfSequence(prod, tmp);
        for (const string& s : tmp) {
          if (firstSets[nt].count(s) == 0) {
            firstSets[nt].insert(s);
            changed = true;
          }
        }
      }
    }
  }
}

void computeFollow() {
  // Initialize
  for (const string& nt : nonTerminals) {
    followSets[nt] = {};
  }
  // Start symbol gets $
  followSets["S"].insert("$");

  bool changed = true;
  while (changed) {
    changed = false;
    for (const string& nt : nonTerminals) {
      for (const vector<string>& prod : grammar[nt]) {
        for (int i = 0; i < (int)prod.size(); i++) {
          const string& sym = prod[i];
          if (!isNonTerminal(sym)) continue;

          // Compute FIRST of the rest: prod[i+1 .. end]
          vector<string> rest(prod.begin() + i + 1, prod.end());
          set<string> firstRest;
          bool hasEps = computeFirstOfSequence(rest, firstRest);

          for (const string& s : firstRest) {
            if (s != "eps") {
              if (followSets[sym].count(s) == 0) {
                followSets[sym].insert(s);
                changed = true;
              }
            }
          }

          // If eps in FIRST(rest), add FOLLOW(nt) to FOLLOW(sym)
          if (hasEps || rest.empty()) {
```

```
                for (const string& s : followSets[nt]) {
                    if (followSets[sym].count(s) == 0) {
                        followSets[sym].insert(s);
                        changed = true;
                    }
                }
            }
        }
    }
}

void printSet(const string& name, const string& nt, const set<string>& s) {
    cout << name << "(" << nt << ") = {";
    bool first = true;
    for (const string& sym : s) {
        if (!first) cout << ", ";
        if (sym == "eps") cout << "e";
        else cout << sym;
        first = false;
    }
    cout << "}" << endl;
}

int main() {
    // Define non-terminals
    nonTerminals = {"S", "A", "B", "C", "D"};
    terminals = {"a", "b", "c", "(", ")", "$"};

    // Define grammar productions
    // S -> A B C | D
    grammar["S"] = {
        {"A", "B", "C"},
        {"D"}
    };
    // A -> a | eps
    grammar["A"] = {
        {"a"},
        {"eps"}
    };
    // B -> b | eps
    grammar["B"] = {
        {"b"},
        {"eps"}
    };
    // C -> ( S ) | c
    grammar["C"] = {
        {"(", "S", ")"},
        {"c"}
    };
```

```
    // D -> A C
    grammar["D"] = {
        {"A", "C"}
    };

    computeFirst();
    computeFollow();

    cout << "=== FIRST Sets ===" << endl;
    // Print in order: S, A, B, C, D
    for (const string& nt : {"S", "A", "B", "C", "D"}) {
        printSet("First", nt, firstSets[nt]);
    }

    cout << endl << "=== FOLLOW Sets ===" << endl;
    for (const string& nt : {"S", "A", "B", "C", "D"}) {
        printSet("Follow", nt, followSets[nt]);
    }

    return 0;
}
```

3.Output:

```
debdootmanna@Debdoots-MacBook-Air DLP % ./7
=== FIRST Sets ===
First(S) = {(, a, b, c}
First(A) = {a, e}
First(B) = {b, e}
First(C) = {(, c}
First(D) = {(, a, c}

=== FOLLOW Sets ===
Follow(S) = {$, )}
Follow(A) = {(, b, c}
Follow(B) = {(, c}
Follow(C) = {$, )}
Follow(D) = {$, )}
debdootmanna@Debdoots-MacBook-Air DLP %
```